

INTRODUCTION TO SYNTAX I

Session 3: Satisfying syntactic requirements via structure building

Timea Szarvas

University of Potsdam

`timea.szarvas@uni-potsdam.de`

July 29th 2026

OVA 2026

TODAY'S PLAN

- ◉ Combining elements into phrases
- ◉ Recursion
- ◉ Phrase structure
- ◉ Phrase types and terminology

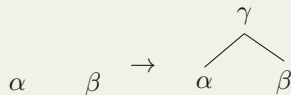
- Predicates drive structure building by c-selecting arguments to satisfy their requirements.

STRUCTURE BUILDING

■ Predicates drive structure building by c-selecting arguments to satisfy their requirements.

■ Structure-building operation Merge

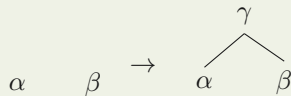
Merge takes **exactly two** constituents α and β and merges them into a new constituent γ .



■ Predicates drive structure building by c-selecting arguments to satisfy their requirements.

■ Structure-building operation Merge

Merge takes **exactly two** constituents α and β and merges them into a new constituent γ .



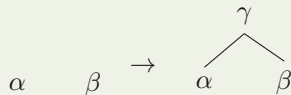
(1) **Full Interpretation**

Syntactic structures must not contain any [uX] features.

■ Predicates drive structure building by c-selecting arguments to satisfy their requirements.

■ Structure-building operation Merge

Merge takes **exactly two** constituents α and β and merges them into a new constituent γ .



(1) Full Interpretation

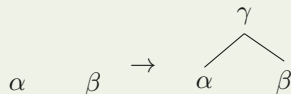
Syntactic structures must not contain any [uX] features.

- ⊙ How do the [uX] features of predicates 'disappear'?

■ **Predicates drive structure building by c-selecting arguments to satisfy their requirements.**

■ Structure-building operation Merge

Merge takes **exactly two** constituents α and β and merges them into a new constituent γ .



(1) **Full Interpretation**

Syntactic structures must not contain any [uX] features.

- ⊙ How do the [uX] features of predicates 'disappear'?

(2) **Feature Checking under sisterhood**

A c-selection feature [uX] on a constituent Y is checked (and thus deleted) if Y is the sister of a constituent Z carrying a matching feature [X].

- ⊙ Notation for checked features: $\{\text{uX}\}$

FEATURE CHECKING: INTRANSITIVE VERBS

(3)

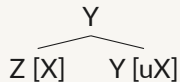
Z [X] Y [uX]

FEATURE CHECKING: INTRANSITIVE VERBS

(3)

Z [X] Y [uX]

(4)

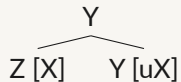


FEATURE CHECKING: INTRANSITIVE VERBS

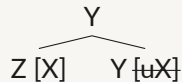
(3)

Z [X] Y [uX]

(4)



(5)

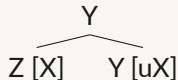


FEATURE CHECKING: INTRANSITIVE VERBS

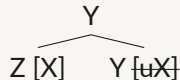
(3)

Z [X] Y [uX]

(4)



(5)



■ Let's build a simple sentence: Predicate: *arrive* [V, uN]; argument: *the train* [N]

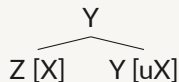
- ⊙ **Full Interpretation:** [uN] on *arrive* must be checked (and deleted) by merging with a constituent carrying a [N] feature.

FEATURE CHECKING: INTRANSITIVE VERBS

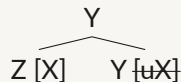
(3)

Z [X] Y [uX]

(4)



(5)



■ Let's build a simple sentence: Predicate: *arrive* [V, uN]; argument: *the train* [N]

- ⊙ **Full Interpretation:** [uN] on *arrive* must be checked (and deleted) by merging with a constituent carrying a [N] feature.

(6)

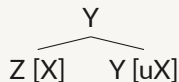
the train arrive
[N] [V, uN]

FEATURE CHECKING: INTRANSITIVE VERBS

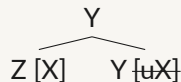
(3)

Z [X] Y [uX]

(4)



(5)



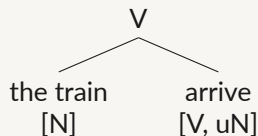
■ Let's build a simple sentence: Predicate: *arrive* [V, uN]; argument: *the train* [N]

- ⊙ **Full Interpretation:** [uN] on *arrive* must be checked (and deleted) by merging with a constituent carrying a [N] feature.

(6)

the train
[N] arrive
 [V, uN]

(7)

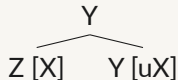


FEATURE CHECKING: INTRANSITIVE VERBS

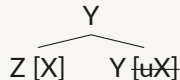
(3)

Z [X] Y [uX]

(4)



(5)



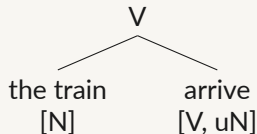
■ Let's build a simple sentence: Predicate: *arrive* [V, uN]; argument: *the train* [N]

- ⊙ **Full Interpretation:** [uN] on *arrive* must be checked (and deleted) by merging with a constituent carrying a [N] feature.

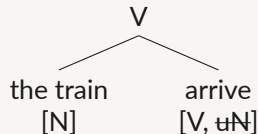
(6)

the train
[N] arrive
[V, uN]

(7)



(8)



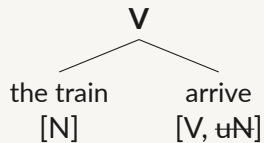
CONSTITUENT LABELS

(9) [_A B C]

(11)



(10)

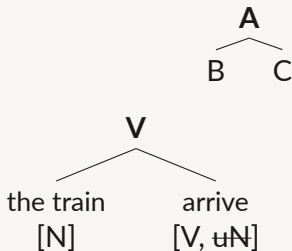


CONSTITUENT LABELS

(9) [_A B C]

(11)

(10)



Complex constituents have labels.

- ⊙ The label represents the properties of a complex constituent.
- ⊙ The label of a constituent is inherited from one of its **daughter nodes**.
- ⊙ The daughter node from which the label inherited to the mother node is the **head** of the constituent.

■ Inflectional features are inherited from head

- (12) a. My friend is cheerful.
b. My parents are cheerful.
c. [The **parents** of my friend] are cheerful.
d. *[The parents of my **friend**] is cheerful.

CONSTITUENTS INHERIT MORPHOSYN. FEATURES OF HEAD

■ Inflectional features are inherited from head

- (12) a. My friend is cheerful.
b. My parents are cheerful.
c. [The **parents** of my friend] are cheerful.
d. *[The parents of my **friend**] is cheerful.

- (13) a. [**Owners** of a pig] love to eat truffles.
b. *[Owners of a **pig**] loves to eat truffles.

CONSTITUENTS INHERIT MORPHOSYN. FEATURES OF HEAD

■ Inflectional features are inherited from head

- (12) a. My friend is cheerful.
b. My parents are cheerful.
c. [The **parents** of my friend] are cheerful.
d. *[The parents of my **friend**] is cheerful.

- (13) a. [**Owners** of a pig] love to eat truffles.
b. *[Owners of a **pig**] loves to eat truffles.

■ PL agreement → *my parents/owners* [PL] are the heads!

CONSTITUENTS INHERIT MORPHOSYN. FEATURES OF HEAD

■ Inflectional features are inherited from head

- (12) a. My friend is cheerful.
b. My parents are cheerful.
c. [The **parents** of my friend] are cheerful.
d. *[The parents of my **friend**] is cheerful.

- (13) a. [**Owners** of a pig] love to eat truffles.
b. *[Owners of a **pig**] loves to eat truffles.

■ PL agreement → *my parents/owners* [PL] are the heads!

■ Category features are inherited from head

- (14) The auxiliary *will* c-selects a verb:
a. I will dance.
b. I will [devour the pizza].
c. *I will the pizza.

CONSTITUENTS INHERIT MORPHOSYN. FEATURES OF HEAD

■ Inflectional features are inherited from head

- (12) a. My friend is cheerful.
b. My parents are cheerful.
c. [The **parents** of my friend] are cheerful.
d. *[The parents of my **friend**] is cheerful.

- (13) a. [**Owners** of a pig] love to eat truffles.
b. *[Owners of a **pig**] loves to eat truffles.

■ PL agreement → *my parents/owners* [PL] are the heads!

■ Category features are inherited from head

(14) The auxiliary *will* c-selects a verb:

- a. I will dance.
b. I will [devour the pizza].
c. *I will the pizza.

(15) The verb *order* c-selects a noun:

- a. I ordered desert.
b. I ordered the pizza.
c. *I ordered [devour the pizza].

CONSTITUENTS INHERIT MORPHOSYN. FEATURES OF HEAD

■ Inflectional features are inherited from head

- (12) a. My friend is cheerful.
b. My parents are cheerful.
c. [The **parents** of my friend] are cheerful.
d. *[The parents of my **friend**] is cheerful.

- (13) a. [**Owners** of a pig] love to eat truffles.
b. *[Owners of a **pig**] loves to eat truffles.

■ PL agreement → *my parents/owners* [PL] are the heads!

■ Category features are inherited from head

(14) The auxiliary *will* c-selects a verb:

- a. I will dance.
b. I will [devour the pizza].
c. *I will the pizza.

(15) The verb *order* c-selects a noun:

- a. I ordered desert.
b. I ordered the pizza.
c. *I ordered [devour the pizza].

■ ⇒ ✓ *will* [V, uV]; ✗ *order* [V, uN] ⇒ *devour* [V] is the head!

■ The head projects its features.

- ⊙ The **head** of a constituent X is the **daughter of X which bears c-selection features**.
 - ⊙ The **label** of the constituent corresponds to the **category feature of the head** (N for noun, V for verb, P for preposition)
- ⇒ All of the morphosyntactic features of the head are projected!

■ The element that c-selects projects!

- ⇒ Heads project their requirements, constraining what syntactic structures can be built.

■ What about heads with more than one c-selection feature?

- ⊙ Let's see how *Hannah devoured the pizza* is derived:¹

¹The process of step-by-step structure building via Merge is called the *derivation*.

■ What about heads with more than one c-selection feature?

- ⊙ Let's see how *Hannah devoured the pizza* is derived:¹

(16)

devour	the pizza
[V, uN, uN]	[N]

¹The process of step-by-step structure building via Merge is called the *derivation*.

FEATURE CHECKING: TRANSITIVE VERBS

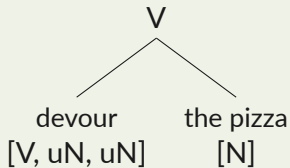
What about heads with more than one c-selection feature?

- Let's see how *Hannah devoured the pizza* is derived:¹

(16)

devour the pizza
[V, uN, uN] [N]

(17)



¹The process of step-by-step structure building via Merge is called the *derivation*.

FEATURE CHECKING: TRANSITIVE VERBS

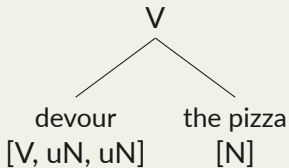
What about heads with more than one c-selection feature?

- Let's see how *Hannah devoured the pizza* is derived:¹

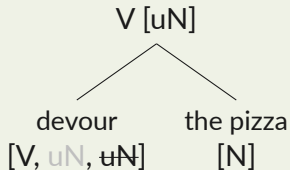
(16)

devour the pizza
[V, uN, uN] [N]

(17)



(18)

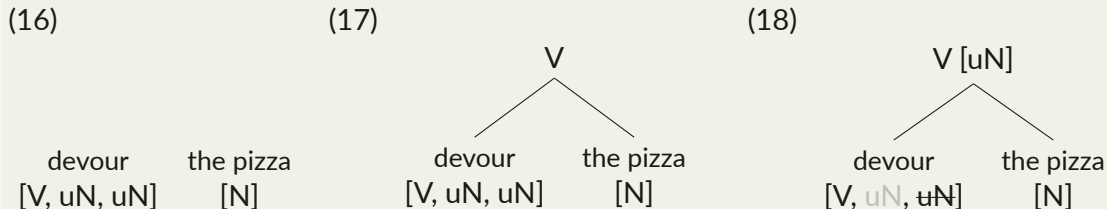


¹The process of step-by-step structure building via Merge is called the *derivation*.

FEATURE CHECKING: TRANSITIVE VERBS

What about heads with more than one c-selection feature?

- Let's see how *Hannah devoured the pizza* is derived:¹



Step 1: *devour* [V, uN, uN] and *the pizza* [N] are inserted from the numeration.

Step 2: Merge combines *devour* with *the pizza*.

Step 3: one c-selectional feature on *devour* [V, uN, uN] is checked+deleted; the other one is inherited to the mother V.

¹The process of step-by-step structure building via Merge is called the *derivation*.

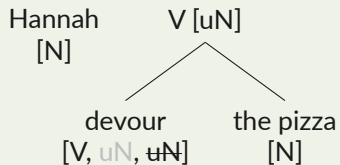
FEATURE CHECKING: TRANSITIVE VERBS

- The head combines with elements until all of its c-selectional features are checked.

FEATURE CHECKING: TRANSITIVE VERBS

- The head combines with elements until all of its c-selectional features are checked.

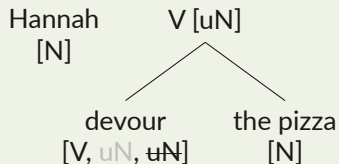
(19)



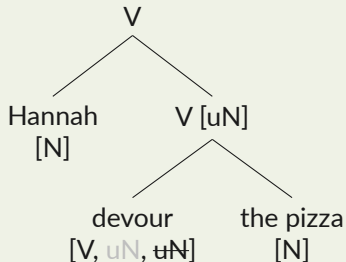
FEATURE CHECKING: TRANSITIVE VERBS

■ The head combines with elements until all of its c-selectional features are checked.

(19)



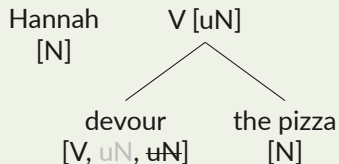
(20)



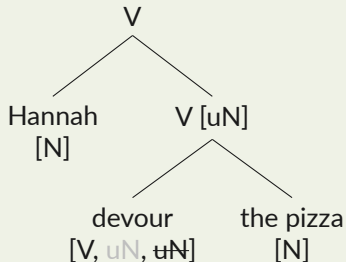
FEATURE CHECKING: TRANSITIVE VERBS

■ The head combines with elements until all of its c-selectional features are checked.

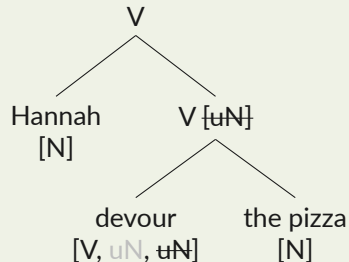
(19)



(20)



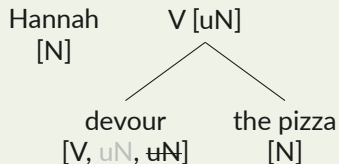
(21)



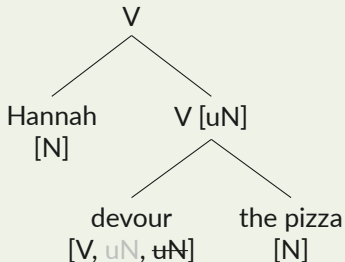
FEATURE CHECKING: TRANSITIVE VERBS

■ The head combines with elements until all of its c-selectional features are checked.

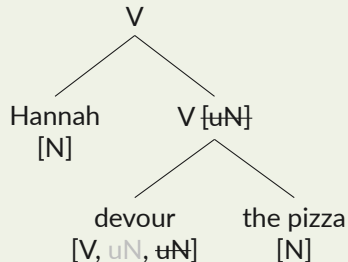
(19)



(20)



(21)



Step 4: *Hannah* is inserted from the numeration.

Step 5: *Hannah* is merged with *devour the pizza* $[uN]$.

Step 6: the c-selectional feature on *devour the pizza* $[uN]$ is checked+deleted $\Rightarrow$ the derivation is complete.

■ Recall session 1:

- ⊙ The two central properties of language we aim to capture via formal syntax are **creativity** and **restrictiveness**.
- ⊙ One of the hallmarks of creativity is **recursion**: a constituent can combine with a constituent to form a more complex constituent, which can combine with yet another constituent to form an even more complex constituent,...

■ Recall session 1:

- ⊙ The two central properties of language we aim to capture via formal syntax are **creativity** and **restrictiveness**.
- ⊙ One of the hallmarks of creativity is **recursion**: a constituent can combine with a constituent to form a more complex constituent, which can combine with yet another constituent to form an even more complex constituent,...
- ⊙ This is precisely what we did when we derived *Hannah devoured the pizza* $\Rightarrow$ [Hannah + [devour + the pizza]].

■ Recall session 1:

- ⊙ The two central properties of language we aim to capture via formal syntax are **creativity** and **restrictiveness**.
- ⊙ One of the hallmarks of creativity is **recursion**: a constituent can combine with a constituent to form a more complex constituent, which can combine with yet another constituent to form an even more complex constituent,...
- ⊙ This is precisely what we did when we derived *Hannah devoured the pizza* $\Rightarrow$ [Hannah + [devour + the pizza]].

■ Formalization via Merge:

- $\Rightarrow$ The complexity of syntactic structures results from repeated applications of Merge.
- $\Rightarrow$ The constituents' selectional criteria constrain what they can Merge with.

■ Two concepts not to be confused!

- ⊙ Verbs are straightforward heads because of their valency requirements.
- ⇒ **Valency** independently requires verbs to combine with a specific number of arguments, to ensure that the expressed event is **semantically meaningful**.

■ Two concepts not to be confused!

- ⊙ Verbs are straightforward heads because of their valency requirements.
- ⇒ **Valency** independently requires verbs to combine with a specific number of arguments, to ensure that the expressed event is **semantically meaningful**.
- ⇒ **Selection** is a broader property of heads in general about the **syntactic category of the elements they can combine with**.

I Two concepts not to be confused!

- ⊙ Verbs are straightforward heads because of their valency requirements.
 - ⇒ **Valency** independently requires verbs to combine with a specific number of arguments, to ensure that the expressed event is **semantically meaningful**.
 - ⇒ **Selection** is a broader property of heads in general about the **syntactic category of the elements they can combine with**.
 - ⊙ In constituents like *the parents of my friend* (12) and *owners of a pig* (13), the heads *the parents* and *owners* select a preposition, but they could also appear without it.
- (22) a. The parents are responsible for the child.
b. Owners are free to choose the tenants they like.

EXERCISE (10 MIN)

(23) the book on the table

① Together:

- ◇ What is the head?
- ◇ What does the head select?
- ◇ What is the category of the whole constituent?

EXERCISE (10 MIN)

(23) the book on the table

① Together:

- ◇ What is the head?
- ◇ What does the head select?
- ◇ What is the category of the whole constituent?

② Individually:

- ◇ What is the structure of the whole constituent? Draw a tree introducing each element step-by-step
- ◇ Once you have the tree: label all of the nodes based on category and selectional features.
- ◇ **Hint 1:** no need to overthink the internal structure of *the book* or *the table*.
- ◇ **Hint 2:** if you have trouble figuring out which elements belong 'closer' together, try the constituency tests from session 1!

I Answers

- ⊙ *the book* is the head
- ⊙ *the book* selects *on the table*
- ⊙ The whole constituent is a noun.

■ Answers

- ⊙ *the book* is the head
- ⊙ *the book* selects *on the table*
- ⊙ The whole constituent is a noun

■ *on the table* is a constituent!

- ⊙ *the book* [*on the table*]/[*in the library*]
- ⊙ it was [***on the table***] that I left the book
- ⊙ *the book* [*on the table*] **and** [*in the library*]

I Answers

- ◉ *the book* is the head
- ◉ *the book* selects *on the table*
- ◉ The whole constituent is a noun

I *on the table* is a constituent!

- ◉ *the book* [*on the table*]/[*in the library*]
- ◉ it was [***on the table***] that I left the book
- ◉ *the book* [*on the table*] **and** [*in the library*]

(24)

on	the table
[P, uN]	[N]

Answers

- *the book* is the head
- *the book* selects *on the table*
- The whole constituent is a noun

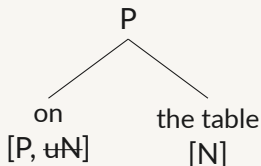
(24)

on the table
[P, uN] [N]

on the table is a constituent!

- *the book* [*on the table*]/[*in the library*]
- it was [***on the table***] that I left the book
- *the book* [*on the table*] **and** [*in the library*]

(25)



Answers

- *the book* is the head
- *the book* selects *on the table*
- The whole constituent is a noun

on the table is a constituent!

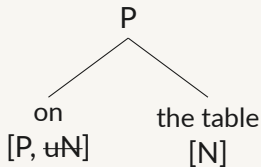
- *the book* [*on the table*]/[*in the library*]
- *it was* [***on the table***] *that I left the book*
- *the book* [*on the table*] **and** [*in the library*]

(24)

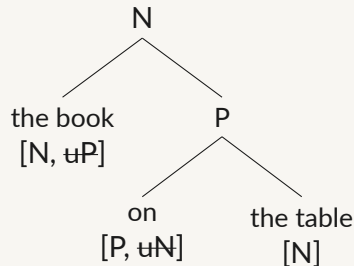
on
[P, uN]

the table
[N]

(25)



(26)



I A quick recap

- ⊙ In session 1, we learned that sentences have a **complex hierarchical structure**.

■ A quick recap

- ⊙ In session 1, we learned that sentences have a **complex hierarchical structure**.
- ⊙ Today, we learned that these complex hierarchical structures are built using the operation **Merge** which **combines two syntactic elements** and can apply repeatedly.

I A quick recap

- ⊙ In session 1, we learned that sentences have a **complex hierarchical structure**.
- ⊙ Today, we learned that these complex hierarchical structures are built using the operation **Merge** which **combines two syntactic elements** and can apply repeatedly.
- ⊙ We also learned that the resulting structures have a **head which projects its features** into the higher structure and thus determines the context in which this structure can appear.

■ A quick recap

- ⊙ In session 1, we learned that sentences have a **complex hierarchical structure**.
- ⊙ Today, we learned that these complex hierarchical structures are built using the operation **Merge** which **combines two syntactic elements** and can apply repeatedly.
- ⊙ We also learned that the resulting structures have a **head which projects its features** into the higher structure and thus determines the context in which this structure can appear.

■ Next, we will learn more about ‘projection’.

- ◉ We have seen that **remaining c-selectional features project**:
⇒ In (18), the remaining [uN] projected from *devour* to V.

²A trivial constituent without c-selection features is simultaneously minimal and maximal!

- ◉ We have seen that **remaining c-selectional features project**:
⇒ In (18), the remaining [uN] projected from *devour* to V.

■ More refined terminology

²A trivial constituent without c-selection features is simultaneously minimal and maximal!

- ◉ We have seen that **remaining c-selectional features project**:
⇒ In (18), the remaining [uN] projected from *devour* to V.

■ More refined terminology

- ◉ Constituents without [uX] features are called **maximal projections** or **phrases**.

²A trivial constituent without c-selection features is simultaneously minimal and maximal!

- ◉ We have seen that **remaining c-selectional features project**:
⇒ In (18), the remaining [uN] projected from *devour* to V.

■ More refined terminology

- ◉ Constituents without [uX] features are called **maximal projections** or **phrases**.
- ◉ Phrases are labeled **XP** (X being the category of the head).

²A trivial constituent without c-selection features is simultaneously minimal and maximal!

- ◉ We have seen that **remaining c-selectional features project**:
⇒ In (18), the remaining [uN] projected from *devour* to V.

■ More refined terminology

- ◉ Constituents without [uX] features are called **maximal projections** or **phrases**.
- ◉ Phrases are labeled **XP** (X being the category of the head).
- ◉ Terminal nodes (that don't branch any further) are called **minimal projections**.²

²A trivial constituent without c-selection features is simultaneously minimal and maximal!

- ◉ We have seen that **remaining c-selectional features project**:
⇒ In (18), the remaining [uN] projected from *devour* to V.

■ More refined terminology

- ◉ Constituents without [uX] features are called **maximal projections** or **phrases**.
- ◉ Phrases are labeled **XP** (X being the category of the head).
- ◉ Terminal nodes (that don't branch any further) are called **minimal projections**.²
- ◉ Nodes that are neither minimal nor maximal (i.e. non-terminal nodes with a [uX] feature) are called **intermediate projections**.

²A trivial constituent without c-selection features is simultaneously minimal and maximal!

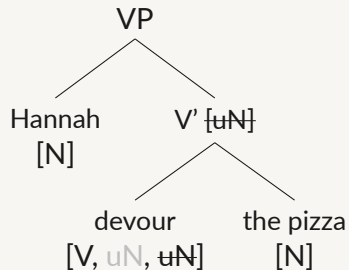
- ◉ We have seen that **remaining c-selectional features project**:
⇒ In (18), the remaining [uN] projected from *devour* to V.

■ More refined terminology

- ◉ Constituents without [uX] features are called **maximal projections** or **phrases**.
- ◉ Phrases are labeled **XP** (X being the category of the head).
- ◉ Terminal nodes (that don't branch any further) are called **minimal projections**.²
- ◉ Nodes that are neither minimal nor maximal (i.e. non-terminal nodes with a [uX] feature) are called **intermediate projections**.
- ◉ Intermediate projections are labeled **X'** (X being the category of the head, X-bar).

²A trivial constituent without c-selection features is simultaneously minimal and maximal!

(27)



- ◉ *N (Hannah)*: minimal and maximal projection
- ◉ *V (devour)*: minimal projection
- ◉ *N (the pizza)*: minimal and maximal projection
- ◉ *V' (devour the pizza)*: intermediate projection
- ◉ *VP (Hannah devour the pizza)*: maximal projection

I First Merge

- ⊙ Implicitly, I showed you that **the smallest possible constituents combine first**:
⇒ In (24-26), *on the table* is built before Merging with *the book*, for example; in (16-18), *devour the pizza* is built before Merging with *Hannah*.

I First Merge

- ⊙ Implicitly, I showed you that **the smallest possible constituents combine first**:
 - ⇒ In (24-26), *on the table* is built before Merging with *the book*, for example; in (16-18), *devour the pizza* is built before Merging with *Hannah*.
- ⊙ Elements that Merge with the head first are called **complements**.

I First Merge

- ⊙ Implicitly, I showed you that **the smallest possible constituents combine first**:
 - ⇒ In (24-26), *on the table* is built before Merging with *the book*, for example; in (16-18), *devour the pizza* is built before Merging with *Hannah*.
- ⊙ Elements that Merge with the head first are called **complements**.
 - ⇒ *the pizza* is the complement of *devour*.
- ⊙ **Merge** combines exactly two elements at a time ⇒ a head has exactly one complement!

I First Merge

- ◉ Implicitly, I showed you that **the smallest possible constituents combine first**:
 - ⇒ In (24-26), *on the table* is built before Merging with *the book*, for example; in (16-18), *devour the pizza* is built before Merging with *Hannah*.
- ◉ Elements that Merge with the head first are called **complements**.
 - ⇒ *the pizza* is the complement of *devour*.
- ◉ **Merge** combines exactly two elements at a time ⇒ a head has exactly one complement!
- ◉ **Complements are maximal projections** (*the pizza* has no [uX]] features) and **sisters of minimal projections** (*devour* is a minimal projection).

I Second Merge

- ⊙ Ok, so *the pizza* Merges with *devour* first to check one of its [uN] features, becoming its complement. What about *Hannah*?

I Second Merge

- ⊙ Ok, so *the pizza* Merges with *devour* first to check one of its [uN] features, becoming its complement. What about *Hannah*?
- ⊙ The intermediate projection *V'* merges with *Hannah* to check its last [uN] feature.

I Second Merge

- ⊙ Ok, so *the pizza* Merges with *devour* first to check one of its [uN] features, becoming its complement. What about *Hannah*?
- ⊙ The intermediate projection *V'* merges with *Hannah* to check its last [uN] feature.
- ⊙ The sister of an intermediate projection is called the **specifier**.

I Second Merge

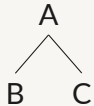
- ◉ Ok, so *the pizza* Merges with *devour* first to check one of its [uN] features, becoming its complement. What about *Hannah*?
- ◉ The intermediate projection *V'* merges with *Hannah* to check its last [uN] feature.
- ◉ The sister of an intermediate projection is called the **specifier**.

I A note on linearization

- ◉ The left-to-right order in which our constituents appeared in the hierarchical structures we saw so far happened to correspond to the order in which they linearly appear in the sentence.
- ⇒ This is not always the case!

(28) [A B C]

(29)



(28) $[_A B C]$

■ **Merge is insensitive to linear order.**

- ⊙ Whether (29)/(28) is linearized as $B \succ C$ or $C \succ B$ depends on language-specific linearization rules!

(29)



(28) $[_A B C]$

■ **Merge is insensitive to linear order.**

(29)



- ◉ Whether (29)/(28) is linearized as $B \succ C$ or $C \succ B$ depends on language-specific linearization rules!
- ◉ Such rules determine whether a complements and specifiers are linearized to the left or right **relative to their sister node**.

(28) $[_A B C]$

■ Merge is insensitive to linear order.

(29)



- ◉ Whether (29)/(28) is linearized as $B \succ C$ or $C \succ B$ depends on language-specific linearization rules!
 - ◉ Such rules determine whether a complement and specifiers are linearized to the left or right **relative to their sister node**.
- ⇒ In other words, a complement is linearized relative to the head (a minimal projection), while a specifier is linearized relative to the [head+complement] (an intermediate projection)!

(28) $[_A B C]$

■ Merge is insensitive to linear order.

(29)



- ◉ Whether (29)/(28) is linearized as $B \succ C$ or $C \succ B$ depends on language-specific linearization rules!
- ◉ Such rules determine whether a complement and specifiers are linearized to the left or right **relative to their sister node**.
- ⇒ In other words, a complement is linearized relative to the head (a minimal projection), while a specifier is linearized relative to the [head+complement] (an intermediate projection)!
- ⇒ **A specifier cannot intrude between a head and its complement!**

Ask away!